Projectory

SW Engineering CSC648/848 Section 06

Team 03

Milestone 2

Architecture UI Mockups and Vertical Software Prototype

October 20, 2026

Member	Role	Primary M2 responsibilities
Rohit Vennelakanti	Team Lead	Coordination, data definitions, 3D integration, testing, documentation
Fanta Phommachith	Scrum Master	Project management, risks, team coordination
Nuradil Kumarov	GitHub Master	Repository, deployment configuration, UML diagrams
Obaidullah Darwishi	AI Master	Functional requirements and AI planning
Ian Ayala	Frontend	React interface, mockups and UX flows
Ali Coklu	Backend	FastAPI, database, authentication and API integration

Document Revision History

Version	Date	Description
1.0	October 20 2026	Initial consolidated M2 documentation
1.1	October 20 2026	Added vertical prototype implementation and team test results
1.2	October 20 2026	Revised schema definitions, recorded instructor feedback, original UX flows and proposed M5 roadmap architecture

1 Data Definitions V2

1.1 Overview

Projectory is a web application that allows students to create digital portfolios showcasing their education, technical skills, professional experience, projects, and supporting media. It also aims to provide AI-assisted career guidance based on a student's portfolio. PostgreSQL through Supabase stores structured application data, while Supabase Storage stores uploaded images and 3D models.

Raw data includes student-entered profiles, descriptions, skills and interests. Metadata includes identifiers, timestamps, file formats, URLs and display order. Supporting data includes authentication information, sharing tokens and external portfolio links. The definitions below use the implementation field names wherever available.

1.2 User Account

A registered student who authenticates and manages an individual portfolio.

Database table: users

Data fields: id: integer primary key; name: text; email: unique text; hashed_password: text; created_at: timestamp.

Registration accepts a name of 1–100 characters, valid email, and a password with at least 8 characters, one letter and one number, and no more than 72 UTF-8 bytes. Password hashes are never part of public responses. An account has a creation timestamp; no separate update timestamp is defined in the inspected model.

1.3 Portfolio

The primary collection of information associated with a student account.

Database table: portfolios

Data fields: id, user_id, headline, bio, location, profile_image_url, career_interests, is_public, share_token, created_at, updated_at.

One portfolio belongs to one user. headline and location allow up to 200 characters, bio 5,000, and profile_image_url 1,000. career_interests is a list of strings. is_public defaults to false; share_token is unique when present. Owners edit their portfolio; visitors only see the permitted public view.

1.4 Education

A student's academic background.

Database table: education

Data fields: id, portfolio_id, school_name, degree, major, start_date, graduation_date, gpa, coursework, created_at, updated_at.

school_name is required and permits up to 200 characters; degree and major also allow 200. Dates use date values. gpa permits 20 characters; coursework is a string list. A portfolio may contain multiple records, which the owner may add, edit or delete.

1.5 Skills

Technical, interpersonal and professional competencies.

Database table: skills

Data fields: id, portfolio_id, name, category, proficiency, created_at, updated_at.

name is required and allows up to 100 characters. Categories are technical, communication, leadership, professional, language or other. proficiency is optional with up to 30 characters. Each portfolio may contain multiple skills.

1.6 Experience

Employment, internships, research, volunteer activities and other relevant work.

Database table: experiences

Data fields: id, portfolio_id, experience_type, organization, position, location, start_date, end_date, is_current, description, created_at, updated_at.

experience_type is work, internship, volunteer, academic, research or other. organization and position are required and allow up to 200 characters. location also permits 200; description permits 5,000. is_current is Boolean. Each portfolio may contain multiple records.

1.7 Projects

Work completed by a student, including descriptive content, technologies, media, links and an optional 3D model.

Database table: projects

Data fields: id, portfolio_id, title, description, content_blocks, project_date, subject, technologies, display_order, created_at, updated_at.

title is required and permits 1–200 characters; description permits 10,000. project_date is a free-form period string of up to 100 characters, such as Fall 2026. subject permits 200. content_blocks contains up to 100 JSON objects used for headings, paragraphs and lists; blocks carry id, type, title and content. technologies is a string list. display_order is integer. The M2 example Raspberry Pi Interactive 3D Scan uses project ID 16.

1.8 Project Media

Supporting images and external links associated with a project.

Database table: project_media

Data fields: id, project_id, media_type, url, label, link_type, storage_path, display_order, created_at.

media_type is image or link. url allows 1,000 characters, label 200, link_type 30, and storage_path 500. Binary files are separate from PostgreSQL. Supported image uploads are JPEG, PNG, GIF and WebP, with a configured default maximum of 10 MB (10 × 1024 × 1024 bytes). Empty or unsupported uploads are rejected. External links must use HTTP or HTTPS. Video upload is not defined; related demonstrations may be linked externally.

1.9 Interactive 3D Models

An interactive model attached to an existing project.

Database table: project_models

Data fields: id, project_id, url, file_format, original_filename, file_size_bytes, storage_path, created_at.

At most one model belongs to a project. url permits 1,000 characters; file_format 10; original_filename 255; storage_path 500. Supported formats are GLB and self-contained glTF with embedded resources; external texture or buffer references in glTF are rejected. The configured default maximum is 50 MB (50 × 1024 × 1024 bytes). Unsupported formats return 400; oversize files return 413. The demonstrated Scaniverse GLB contains 2,217,516 bytes, approximately 2.22 MB. The frontend supports rotation and zoom through Three.js, React Three Fiber and Drei. Frontend upload controls remain planned for M5.

1.10 Career Guidance

Saved analysis of a student's portfolio, generated by the configured guidance provider.

Database table: career_guidance

Data fields: id, portfolio_id, summary, career_paths, strengths, improvements, next_steps, provider, model, created_at.

summary is text. career_paths contains title and explanation; strengths contains title and explanation; improvements contains area and explanation; next_steps contains action, category and supports. A next-step category is skills, projects, experience or career_exploration. provider and model record generation metadata. A refresh adds a new record so history remains available. A live external AI workflow remains unverified for M2; the inspected configuration supports a mock provider and an Anthropic provider.

1.11 Additional Supporting Data

Table	Purpose and data definition
portfolio_links	id, portfolio_id, label, url. External profiles and websites; label is required up to 100 characters and URL up to 1,000.
certificates	id SERIAL NOT NULL PRIMARY KEY; portfolio_id INTEGER NOT NULL REFERENCES portfolios(id) ON DELETE CASCADE; name VARCHAR(200) NOT NULL; issuer VARCHAR(200), issue_year INTEGER and credential_url VARCHAR(1000) nullable; created_at and updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(). Index: ix_certificates_portfolio_id (non-unique). Multiple certificates may belong to one portfolio.
job_preferences	id SERIAL NOT NULL PRIMARY KEY; portfolio_id INTEGER NOT NULL REFERENCES portfolios(id) ON DELETE CASCADE; role VARCHAR(200) NOT NULL; location VARCHAR(200), employment_type VARCHAR(30) and min_salary INTEGER nullable; created_at and updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(). Index: ix_job_preferences_portfolio_id (non-unique). Multiple preferences may belong to one portfolio; these records remain private. The SQL schema does not specify a salary currency, pay period, employment-type enumeration or minimum-salary CHECK constraint.

The supplied PostgreSQL schema defines certificates and job_preferences as portfolio-owned records. Their exact columns and constraints are listed in the preceding table. SQL nullability and constraints are separate from application-level validation.

1.12 Relationships and Access Control

One user has at most one portfolio through unique portfolios.user_id. A portfolio may contain many education, skill, experience, project, link, certificate, job preference and guidance records. A project may have many media records and at most one model. All child foreign keys in the supplied schema use ON DELETE CASCADE. Tag-list limits described above are application validation rules rather than SQL CHECK constraints.

Authenticated owners may manage their own records. Public visitors cannot edit or delete records. Ownership and visibility must be enforced by FastAPI, including requests made by another authenticated user. Public portfolio responses omit the owner's email and account settings. Job preferences must remain private. Public asset URLs require separate privacy review because hiding a portfolio alone does not revoke a known storage URL.

2 Functional Requirements V2

2.1 Overview

The following requirements preserve the R1–R17 organization and numbering from M1 and the supplied M2 requirements. Priority 1 means must have, 2 means desired, and 3 means opportunistic. Priorities describe the intended complete application; they do not assert that every requirement was implemented by M2.

R1 User Registration

ID	Functional requirement	Priority
R1.1	The system shall allow students to register using their name, email address, and password.	1
R1.2	The system shall verify that the email address is not already registered.	1
R1.3	The system shall require all mandatory registration fields to be completed.	1
R1.4	The system shall validate that passwords meet the minimum security requirements.	1
R1.5	The system shall display a confirmation message after successful registration.	2

R2 User Login and Logout

ID	Functional requirement	Priority
R2.1	The system shall allow registered users to log in using their email address and password.	1
R2.2	The system shall verify login credentials before granting access to a user's account.	1
R2.3	The system shall display an error message when invalid login credentials are provided.	1
R2.4	The system shall allow authenticated users to log out of their accounts.	1
R2.5	The system shall prevent unauthorized modification of another student's portfolio, including attempts by authenticated users who are not the owner.	1

R3 Personal Information Management

ID	Functional requirement	Priority
R3.1	The system shall allow students to add personal information, including their name and a short biography, to their portfolio.	1
R3.2	The system shall allow students to view their saved personal information.	1
R3.3	The system shall allow students to edit their personal information.	1
R3.4	The system shall allow students to add and update their career interests.	1
R3.5	The system shall allow students to include links to external professional profiles or personal websites.	2

R4 Education Management

ID	Functional requirement	Priority
R4.1	The system shall allow students to add education records to their portfolio.	1
R4.2	The system shall allow students to enter the school name, degree program, major, and expected or completed graduation date.	1
R4.3	The system shall allow students to edit existing education records.	1
R4.4	The system shall allow students to delete education records.	1
R4.5	The system shall allow students to add relevant coursework to an education record.	2

R5 Skills Management

ID	Functional requirement	Priority
R5.1	The system shall allow students to add skills to their portfolio.	1
R5.2	The system shall allow students to edit existing skills.	1
R5.3	The system shall allow students to remove skills from their portfolio.	1
R5.4	The system shall display the student's saved skills within their portfolio.	1
R5.5	The system shall allow students to categorize their skills into groups such as technical, communication, or other professional skills.	2

R6 Experience Management

ID	Functional requirement	Priority
R6.1	The system shall allow students to add work, internship, volunteer, academic, or other relevant experience.	1
R6.2	The system shall allow students to enter the organization name, position, dates, and description of their responsibilities or accomplishments.	1
R6.3	The system shall allow students to edit existing experience records.	1
R6.4	The system shall allow students to delete experience records.	1
R6.5	The system shall display multiple experience records within a student's portfolio.	1

R7 Project Management

ID	Functional requirement	Priority
R7.1	The system shall allow students to create a project entry in their portfolio.	1
R7.2	The system shall allow students to enter a project title and description.	1

ID	Functional requirement	Priority
R7.3	The system shall allow students to specify technologies or skills used in a project.	2
R7.4	The system shall allow students to edit existing project information.	1
R7.5	The system shall allow students to view all saved projects within their portfolio.	1

Proposed addition for team review: R7.6 — Allow a portfolio owner to delete an existing project. Priority 1. This carries forward the deletion capability already described in M1.

R8 Project Media

ID	Functional requirement	Priority
R8.1	The system shall allow students to attach supported images to a project.	2
R8.2	The system shall allow students to add external links to project repositories, demonstrations, or related resources.	2
R8.3	The system shall display attached images and links within the corresponding project entry.	2
R8.4	The system shall allow students to remove attached project images and links.	2

R9 Portfolio Display

ID	Functional requirement	Priority
R9.1	The system shall display a student's personal information and career interests in their portfolio.	1
R9.2	The system shall organize education, skills, and experience into separate sections.	1
R9.3	The system shall display saved projects in a dedicated portfolio section.	1
R9.4	The system shall display updated portfolio information after a student saves changes.	1
R9.5	The system shall display project media when supported media has been attached.	2

R10 AI Portfolio Analysis

ID	Functional requirement	Priority
R10.1	The system shall retrieve the student's saved education, skills, experience, career interests, and project information for AI analysis.	1
R10.2	The system shall submit relevant portfolio information to the configured AI service through the backend.	1
R10.3	The system shall generate an AI-based summary of the student's academic and professional background.	1
R10.4	The system shall display the generated analysis to the student.	1
R10.5	The system shall notify the student when an AI analysis cannot be completed.	1

R11 Career Path Recommendations

ID	Functional requirement	Priority
R11.1	The system shall use the student's portfolio information and career interests to generate possible career path recommendations.	1
R11.2	The system shall display the recommended career paths to the student.	1
R11.3	The system shall provide a brief explanation of how each recommended career path relates to the student's background.	1
R11.4	The system shall allow students to view multiple recommended career paths when available.	2

R12 Strength Identification

ID	Functional requirement	Priority
R12.1	The system shall analyze the student's portfolio information to identify potential strengths.	1
R12.2	The system shall consider the student's education, skills, experience, and projects when identifying strengths.	1
R12.3	The system shall display identified strengths in the career guidance results.	1
R12.4	The system shall provide a brief explanation connecting each identified strength to relevant portfolio information.	2

R13 Improvement Recommendations

ID	Functional requirement	Priority
R13.1	The system shall identify possible areas for improvement based on the student's portfolio information.	1
R13.2	The system shall generate suggested next steps related to developing skills, improving projects, gaining experience, or exploring careers.	1
R13.3	The system shall display improvement areas and suggested next steps within the career guidance results.	1
R13.4	The system shall explain how a suggested improvement or next step may support a possible career direction.	2

R14 Career Analysis Refresh

ID	Functional requirement	Priority
R14.1	The system shall allow students to request a new AI career analysis after updating their portfolio.	2
R14.2	The system shall use the student's currently saved portfolio information when generating a refreshed analysis.	2
R14.3	The system shall display the updated career guidance results after a successful refresh.	2
R14.4	The system shall display an appropriate error message when the refresh request fails.	2

R15 3D Model Attachment

ID	Functional requirement	Priority
R15.1	The system shall allow students to attach a supported interactive 3D model to an existing portfolio project.	2
R15.2	The system shall accept only supported 3D model formats or assets compatible with the selected 3D viewer.	2
R15.3	The system shall associate an attached 3D model with the selected project.	2
R15.4	The system shall allow students to remove a previously attached 3D model from a project.	2
R15.5	The system shall display an error message when an unsupported 3D model is submitted.	2

R16 Interactive 3D Model Viewing

ID	Functional requirement	Priority
R16.1	The system shall display a supported 3D model within its associated project page.	2
R16.2	The system shall allow portfolio viewers to rotate a displayed 3D model.	2
R16.3	The system shall allow portfolio viewers to zoom in and out of a displayed 3D model.	2
R16.4	The system shall display an appropriate message when a 3D model cannot be loaded.	2
R16.5	The system shall allow viewers to reset a model to its default viewing position.	3

R17 Portfolio Sharing

ID	Functional requirement	Priority
R17.1	The system shall allow students to generate or access a shareable link to their portfolio.	2
R17.2	The system shall allow visitors to open a shared portfolio without logging into a student account.	2
R17.3	The system shall display the student's shared background, education, skills, experience, and projects.	2
R17.4	The system shall allow visitors to view supported project media and interactive 3D models when available.	2
R17.5	The system shall prevent visitors from editing or deleting information in a shared portfolio.	2

R18 Student Approved Career Roadmaps and Milestone Tracking

Proposed addition for M5 team review, priority 2. This requirement was not implemented or verified in M2 and does not change the recorded R1–R17 requirements.

R18.1 The system shall allow the student to discuss portfolio-based career goals with an AI chatbot and request a proposed roadmap.

R18.2 The system shall allow the chatbot to propose creation or editing of a roadmap and individual goals or milestones, showing the proposed changes for review.

R18.3 The system shall save proposed changes only after explicit student approval; rejecting a proposal shall preserve the saved plan.

R18.4 The system shall allow the owner to view and edit approved goals and milestones and mark milestones complete or incomplete.

R18.5 The system shall persist approved plans and completion status and display progress after reload.

R18.6 The system shall enforce ownership for roadmap and milestone operations and report generation or save failures without claiming success.

2.2 M2 Implementation Status

Team testing verified registration and authentication; project creation and editing; persistence in PostgreSQL; database retrieval and project search; backend GLB upload; Supabase Storage persistence; and interactive browser rendering with rotation and zoom. These are team-reported M2 demonstration results, supported by the included screenshots and request results.

Work toward M5 includes complete live AI analysis and proposed student-approved roadmaps and milestone tracking, frontend 3D upload controls, project visibility and sharing validation, remaining portfolio end-to-end tests, error handling, accessibility and usability improvements.

3 UI Mockups and UX Flows

3.1 Overview

The React frontend uses navigation, portfolio forms, project cards and interactive media. This section preserves actual frontend screenshots and adds three original hand-drawn UX flow diagrams. The drawings describe navigation and user actions; they are not detailed screen wireframes and do not establish implementation completion.

The implemented screenshots below document the M2 interface. Section 3.9 contains the original flow diagrams, including intended sharing and career-guidance paths that still require implementation or validation.

3.2 Registration and Login

Requirements R1 and R2. As a student, I want to register and log in so that I can manage my portfolio. Flow: Home → Sign Up → Enter name, email and password → Create Account → Account Setup → Portfolio. Returning users follow Home → Login → Enter Credentials → Authenticated Portfolio. Invalid credentials should return a clear error and keep the user on the form.

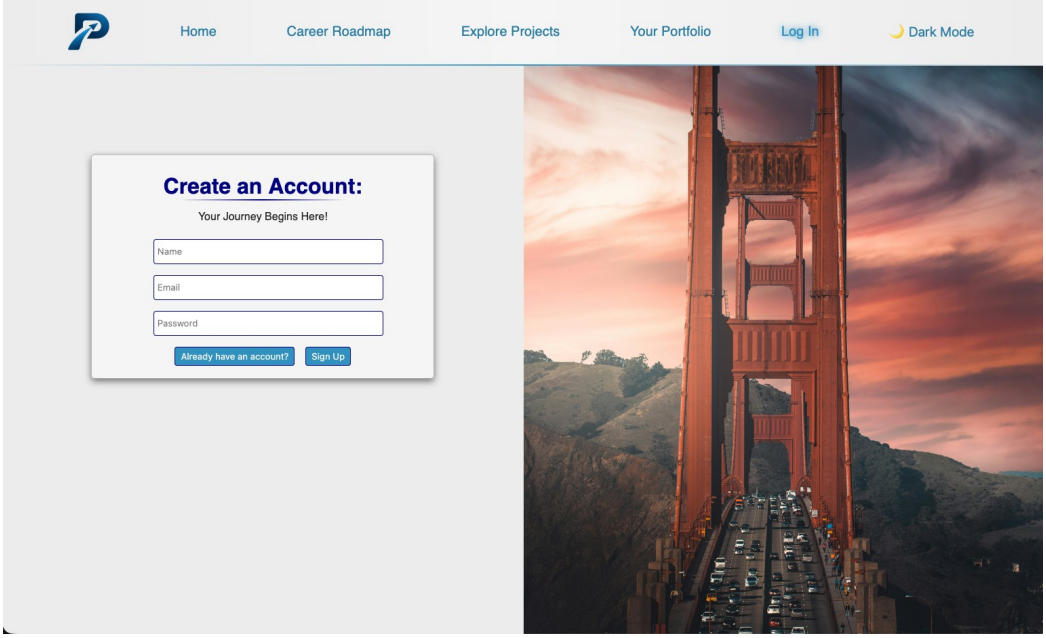


Figure 3.1 Actual Create an Account screen

The separate login and invalid-credentials states are not pictured; their behavior remains part of the end-to-end validation plan.

3.3 Portfolio Creation and Editing

Requirements R3 and R9. As a student, I want my portfolio to accurately represent my background. Flow: Login → Account Setup or Profile Editor → Modify personal information and background → Save Profile → View updated portfolio. The save action should give understandable confirmation and preserve saved values after reload.

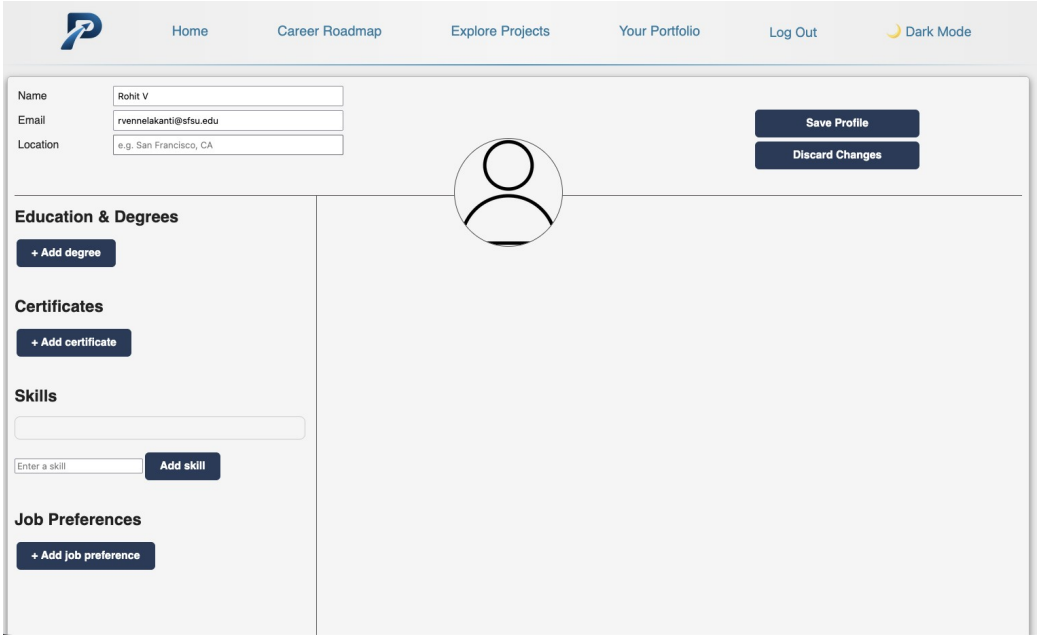


Figure 3.2 Actual profile setup with education skills certificates and preferences

3.4 Project Creation and Management

Requirements R7 and R8. As a student, I want to add and manage projects to demonstrate my work. Flow: Your Portfolio → Create New Project → Enter title, date, subject and content → Add images or external links → Save Project → View saved project → Edit. Errors should identify invalid fields without losing entered content.

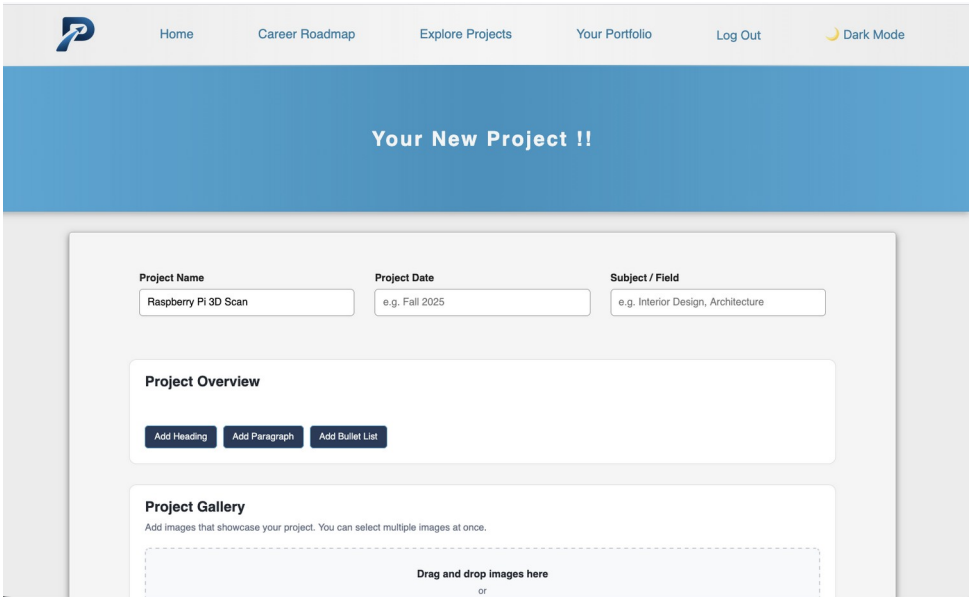


Figure 3.3 Actual project creation form and content editor

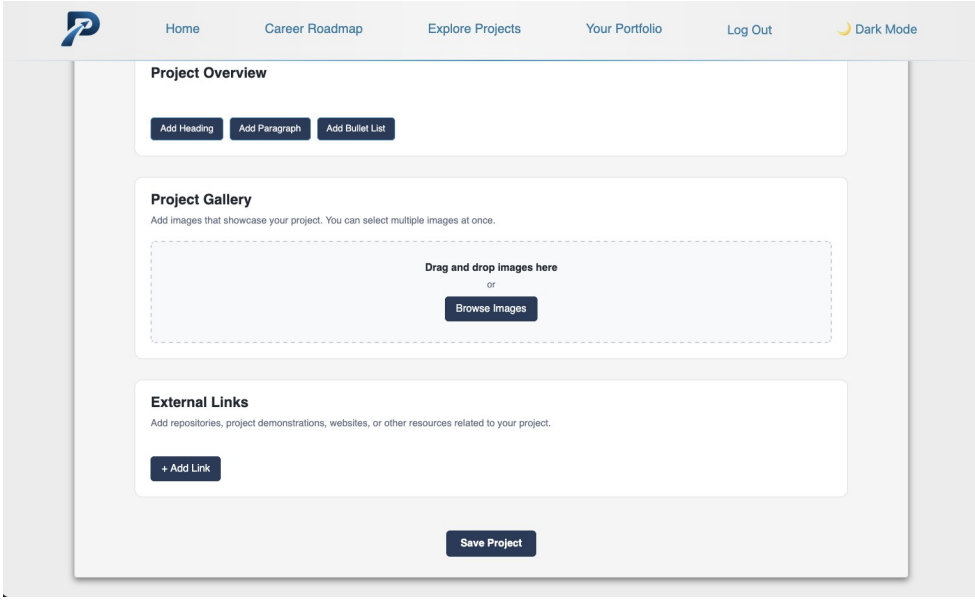


Figure 3.4 Actual project gallery external links and Save Project controls

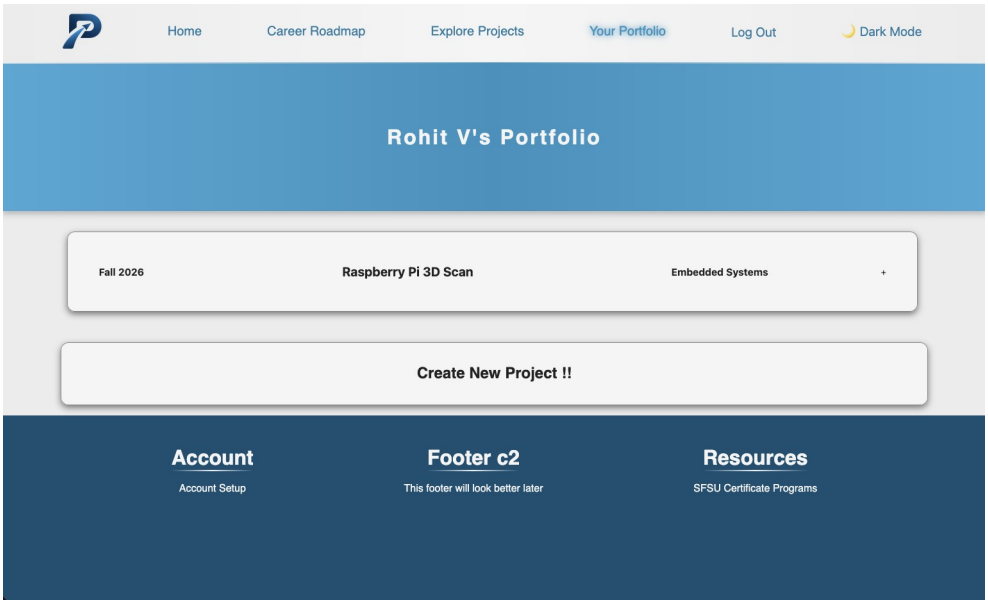


Figure 3.5 Actual saved project in the owner portfolio

3.5 Project Discovery

Requirements R7 and R9, with an M2 search extension. As a visitor, I want to discover student projects related to my interests. Flow: Home → Explore Projects → Enter keyword and optional category → Search → View matching cards → Expand project. Searching robot retrieves Line Following Robot. The list is retrieved from stored records rather than a frontend placeholder array.

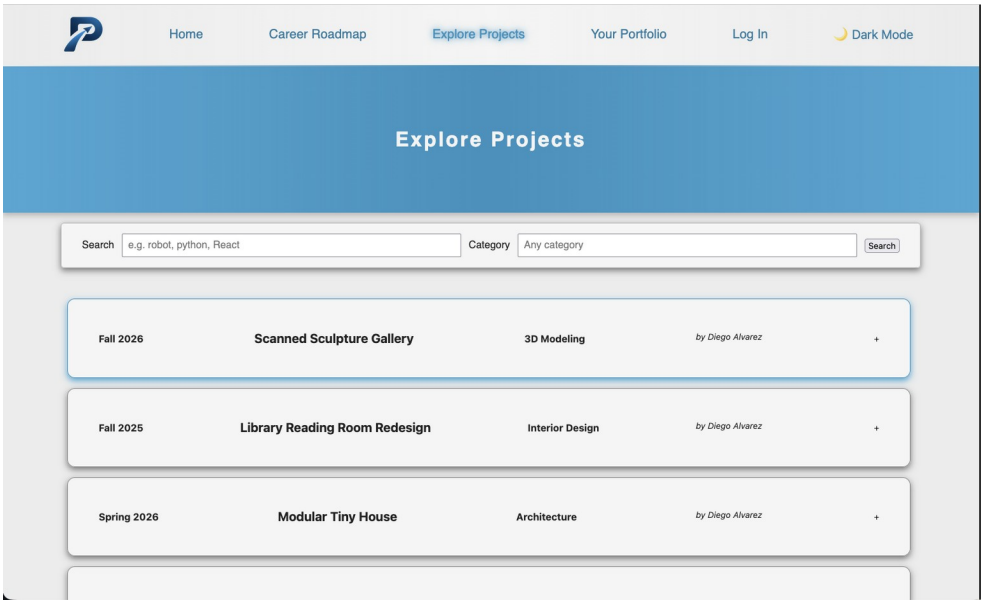


Figure 3.6 Actual database-backed Explore Projects listing

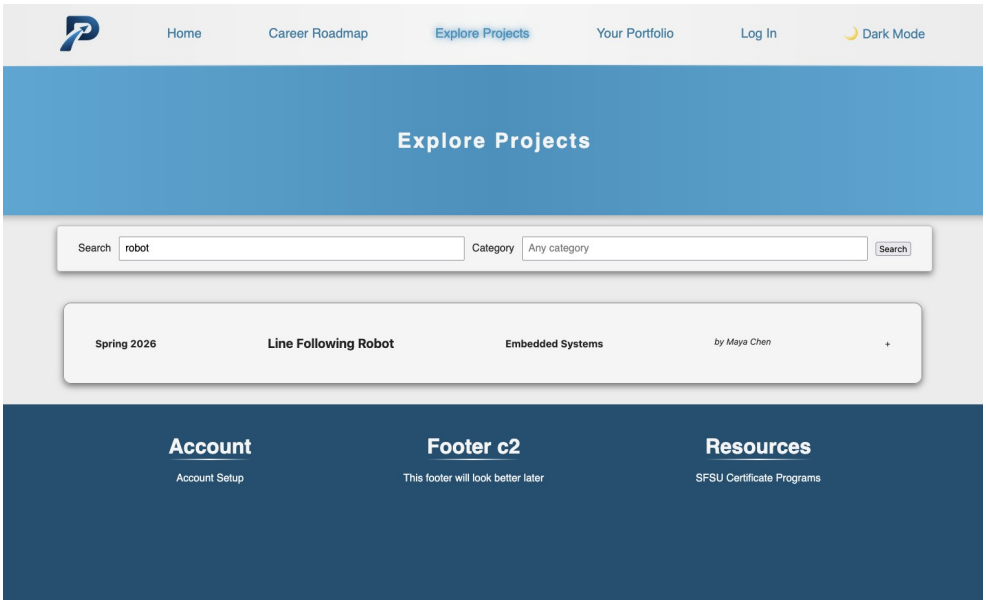


Figure 3.7 Actual robot search result showing Line Following Robot

3.6 Interactive 3D Viewing

Requirements R15 and R16. As a viewer, I want to explore a physical project interactively. Flow: Open Portfolio → Expand Project → Load Associated Model → Rotate or Zoom → Continue Viewing. M2 attaches the model through authenticated Swagger UI; the normal frontend file-selection flow remains planned for M5.

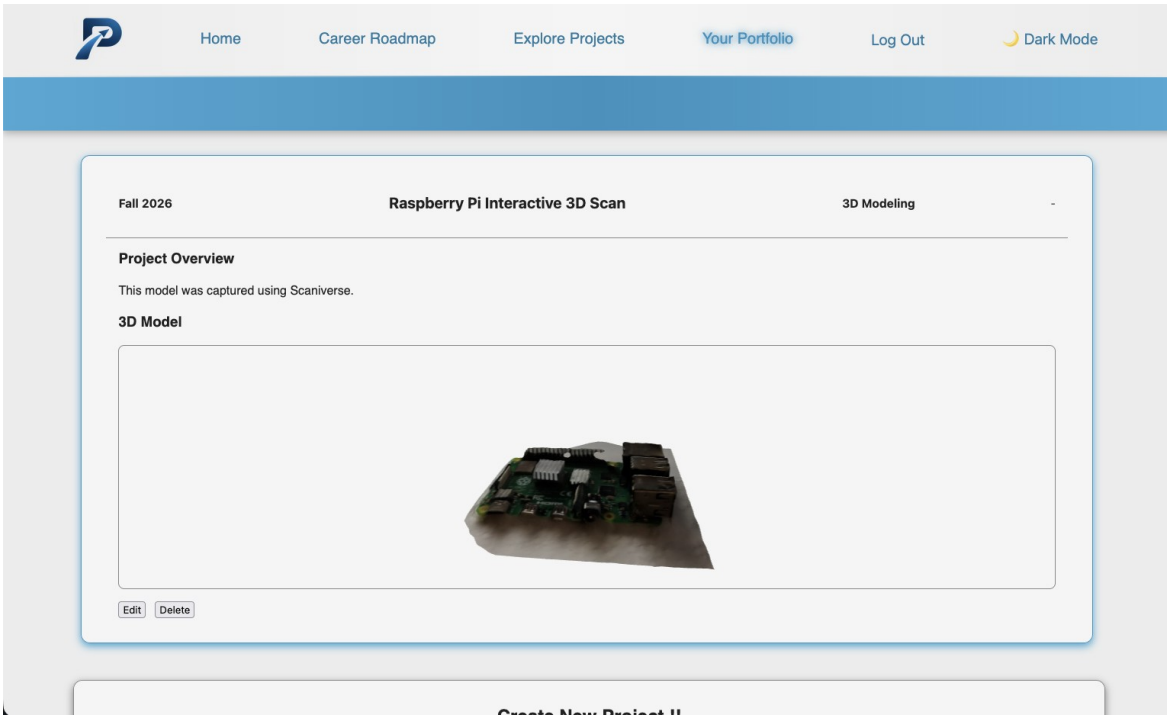


Figure 3.8 Actual Raspberry Pi scan rendered inside the deployed project card

3.7 AI Career Guidance

Requirements R10–R14 and proposed R18. As a student, I want guidance based on my portfolio and control over my career plan. Planned flow: Login → Review Portfolio → Open Career Guidance → Request new guidance or view saved guidance → Chat about career interests → Preview proposed roadmap, goals and milestones → Approve or revise → Save → Track completion. Creation and editing through the chatbot require student approval before persistent changes. Loading, success, failure and retry states require validation. Live external AI analysis and the roadmap workflow were not verified as M2 features.

The hand-drawn career-guidance flow in Figure 3.11 shows new and saved guidance. It does not depict the proposed chatbot, approval controls or milestone tracking; detailed screen designs for those M5 features remain planned.

Related sharing flow for R17: Owner enables sharing → Copy share URL → Visitor opens URL → Backend validates share token and visibility → Read-only portfolio. Disabling sharing should prevent subsequent public retrieval.

The hand-drawn sharing flow in Figure 3.10 describes intended navigation. Public-sharing and disabled-share states still need end-to-end validation; individual-project sharing in the drawing is a design intention, not proof of a separate implemented sharing endpoint.

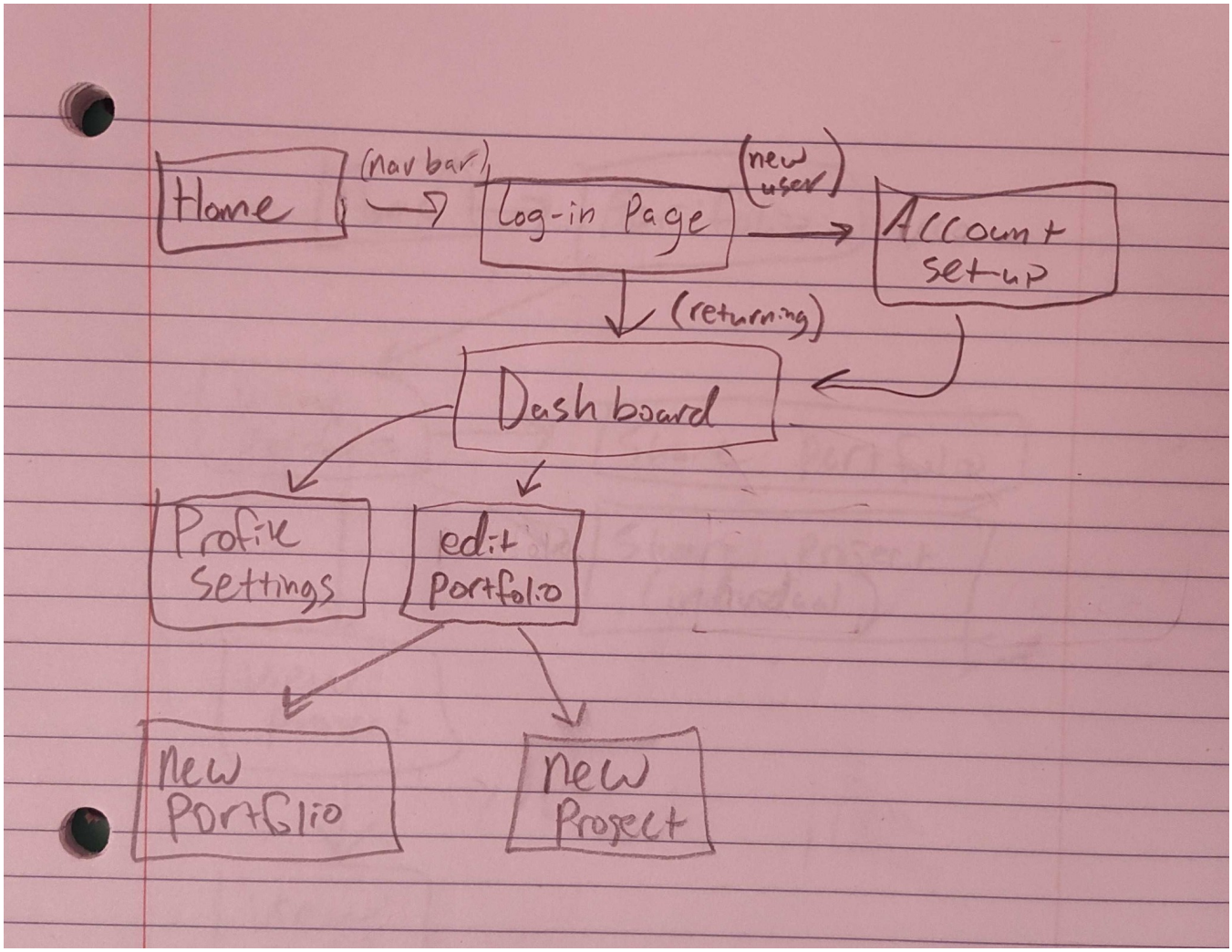
3.8 UX Principles Evaluation

Principle	Design approach and remaining evaluation
Useful	Portfolio organization, project discovery and planned career guidance support student goals.
Usable	Recognizable forms and separate discovery and owner views; verify save confirmations and recovery paths.
Desirable	Project cards and interactive 3D present student work visually.
Accessible	Readable organization; keyboard focus, contrast, labels and assistive-technology behavior require testing.
Findable	Navigation connects portfolio and Explore; keyword and category filters locate projects.

Principle	Design approach and remaining evaluation
Credible	Persistent account-based records support trust; live AI results must clearly reflect their provider and limitations.

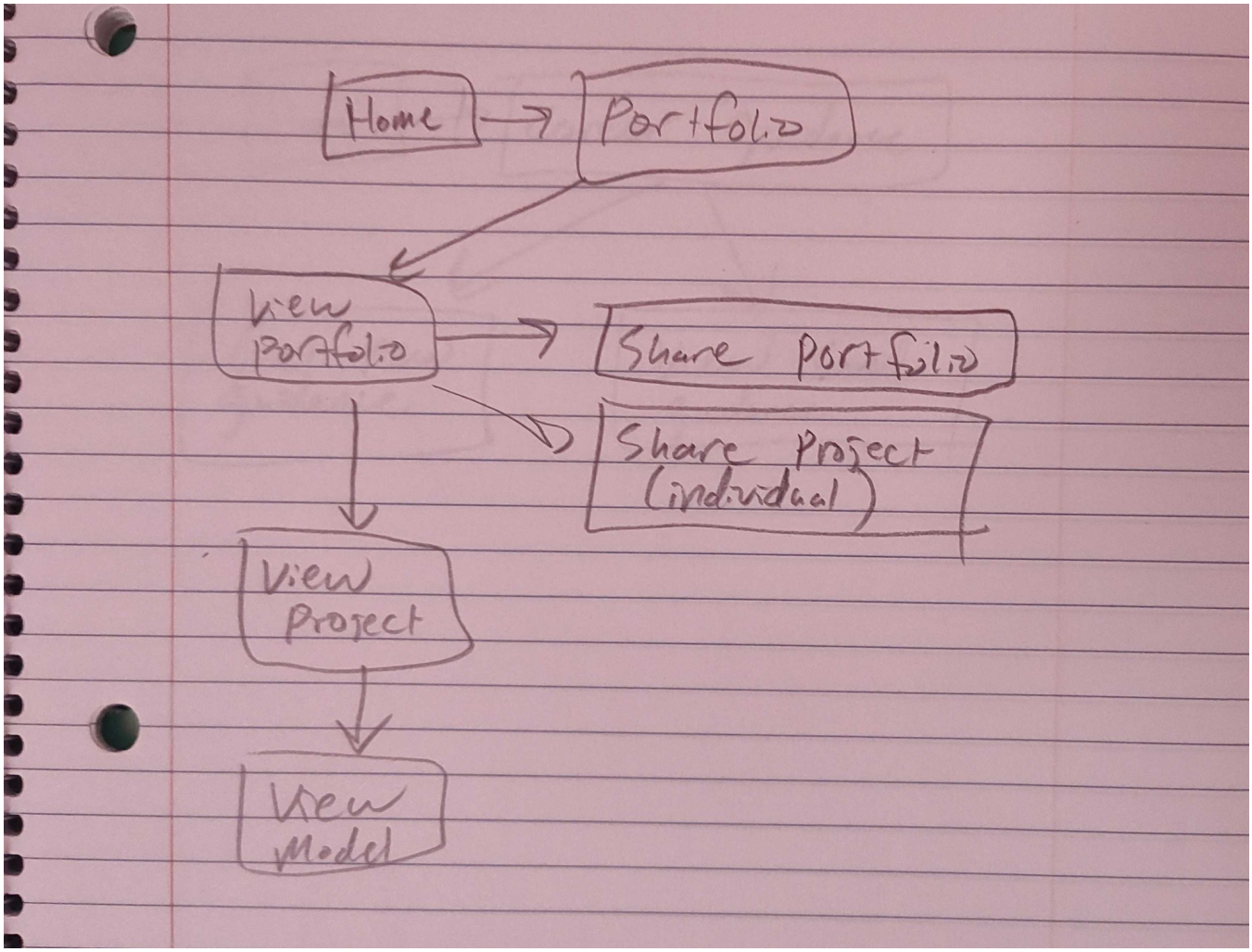
3.9 Original Hand Drawn UX Flow Diagrams

Figure 3.9 Original hand-drawn account setup and portfolio editing flow



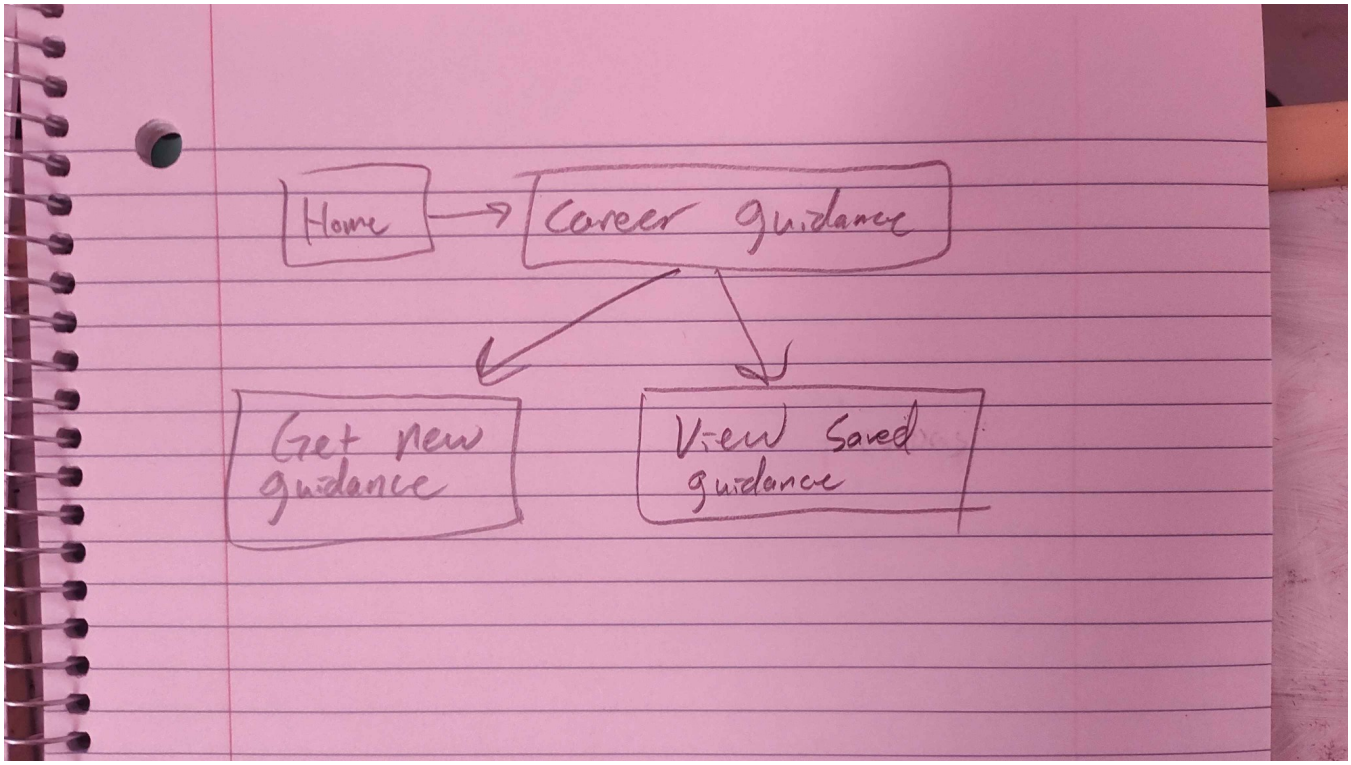
Home leads to login; new users continue through account setup while returning users reach the dashboard. The dashboard branches to profile settings and portfolio editing, then new portfolio or new project.

Figure 3.10 Original hand-drawn portfolio viewing and sharing flow



Home leads to Portfolio and View Portfolio, with branches to Share Portfolio, Share Project (individual), and View Project → View Model. These are intended UX paths; sharing behavior still requires validation.

Figure 3.11 Original hand-drawn career guidance flow



Home leads to Career Guidance, which branches to Get New Guidance and View Saved Guidance. The expanded chatbot and roadmap interaction is proposed separately for M5.

4 High Level Architecture and Database Organization

4.1 Architecture Overview

Projectory follows a layered architecture. The browser renders the React application hosted on Vercel. HTTP requests reach Python FastAPI on Render, which validates input, checks ownership, reads and writes PostgreSQL through Supabase, and returns structured JSON. Supabase Storage holds uploaded media; PostgreSQL stores their URLs and project relationships.

Architecture flow: Browser → React on Vercel → FastAPI on Render → Supabase PostgreSQL. Media flow: FastAPI → Supabase Storage; browser viewer → stored model URL. Planned AI flow: FastAPI reads portfolio → configured AI provider → structured results stored in PostgreSQL → React guidance view.

4.2 Technology Stack

Component	Technology	Purpose
Frontend	React and Vite	Pages forms and project cards
Frontend hosting	Vercel	Deploy static app and SPA routes
Backend	Python FastAPI Pydantic SQLAlchemy	Validation API ownership and database access
Backend hosting	Render	Host application API
Database	Supabase PostgreSQL	Persistent application records
Media	Supabase Storage	Persist images and 3D files
3D	Three.js React Three Fiber Drei	WebGL rendering glTF loading camera controls
API documentation	OpenAPI Swagger UI	Inspect and exercise endpoints
Version control	Git and GitHub	Branches pull requests milestone assets

Relative to M1, the browser viewer selection is now Three.js with React Three Fiber and Drei. Scaniverse is used externally to capture and export the sample asset; no Niantic Spatial SDK is required by the demonstrated viewer. Open-source components remain separated by responsibility. The inspected AI configuration supports mock guidance and an Anthropic provider; provider selection and live API credentials require validation before M5.

4.3 Database Organization

The supplied PostgreSQL schema contains twelve tables: users, portfolios, career_guidance, certificates, education, experiences, job_preferences, portfolio_links, projects, skills, project_media and project_models. It is reference documentation generated from app/models.py; the application creates its tables on startup. JSON collections use JSONB in PostgreSQL. The inventory below records the current schema; proposed M5 roadmap tables in Section 4.8 are outside these twelve tables.

users

id: integer primary key; name: text; email: unique text; hashed_password: text; created_at: timestamp.

portfolios
id, user_id, headline, bio, location, profile_image_url, career_interests, is_public, share_token, created_at, updated_at.

education
id, portfolio_id, school_name, degree, major, start_date, graduation_date, gpa, coursework, created_at, updated_at.

skills
id, portfolio_id, name, category, proficiency, created_at, updated_at.

experiences
id, portfolio_id, experience_type, organization, position, location, start_date, end_date, is_current, description, created_at, updated_at.

projects
id, portfolio_id, title, description, content_blocks, project_date, subject, technologies, display_order, created_at, updated_at.

project_media
id, project_id, media_type, url, label, link_type, storage_path, display_order, created_at.

project_models
id, project_id, url, file_format, original_filename, file_size_bytes, storage_path, created_at.

career_guidance
id, portfolio_id, summary, career_paths, strengths, improvements, next_steps, provider, model, created_at.

portfolio_links
id, portfolio_id, label, url. Foreign key to portfolios.

certificates and job_preferences
certificates: id SERIAL NOT NULL PRIMARY KEY; portfolio_id INTEGER NOT NULL REFERENCES portfolios(id) ON DELETE CASCADE; name VARCHAR(200) NOT NULL; issuer VARCHAR(200), issue_year INTEGER and credential_url VARCHAR(1000) nullable; created_at and updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(). Index: ix_certificates_portfolio_id (non-unique). Multiple certificates may belong to one portfolio.
job_preferences: id SERIAL NOT NULL PRIMARY KEY; portfolio_id INTEGER NOT NULL REFERENCES portfolios(id) ON DELETE CASCADE; role VARCHAR(200) NOT NULL; location VARCHAR(200), employment_type VARCHAR(30) and min_salary INTEGER nullable; created_at and updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(). Index: ix_job_preferences_portfolio_id (non-unique). Multiple preferences may belong to one portfolio; these records remain private. The SQL schema does not specify a salary currency, pay period, employment-type enumeration or minimum-salary CHECK constraint.

All primary keys are SERIAL integers. users.email, portfolios.user_id, portfolios.share_token and project_models.project_id have unique indexes; the remaining child portfolio_id and project_id indexes are non-unique. PostgreSQL permits multiple NULL share_token values. Each child foreign key uses ON DELETE CASCADE. Portfolio, education, skill, experience, project, certificate and job-preference records have created_at and updated_at; users, career_guidance, project_media and project_models have created_at only. portfolio_links has no timestamp columns. Timestamp columns shown in the schema are NOT NULL with DEFAULT now(); that default alone does not automatically update updated_at on every edit.

4.4 Add Delete Search and Update Architecture

Entity	Operations and fields
users	Register name email password; find by email for login; display current user. No user-deletion endpoint documented.
portfolios	Read owner profile; update name headline bio location image URL interests; enable or disable sharing by token.
education	Add list update delete by record ID with school degree major dates GPA coursework.
skills	Add list update delete by record ID with name category proficiency.
experiences	Add list update delete by record ID with type organization position location dates current flag description.
projects	Create read update delete by project ID; list owner records; search title description subject technologies; filter subject and paginate.
project_media	Attach image upload image URL or external link; display by project; remove media by ID.
project_models	Upload or replace by project ID; retrieve metadata with project; remove model.
portfolio_links	Add list update delete label and URL by link ID.
certificates and job_preferences	Portfolio-owned certificate and private job-preference records; exact SQL columns and constraints are in Sections 1.11 and 4.3. Reported CRUD routes still require end-to-end verification.
career_guidance	Generate from owner portfolio; list history; retrieve latest. No editing endpoint documented.

4.5 Major Backend APIs

API base: https://careergps-backend-n1uj.onrender.com. The implemented portfolio prefix is singular /portfolio/me. Owner-specific requests require a Bearer token. JSON is used for ordinary bodies; uploaded files use multipart/form-data.

Endpoint	Contract and purpose
POST /auth/signup	name email password → account and token
POST /auth/login; POST /auth/token	JSON credentials or OAuth form → Bearer token
POST /auth/logout; GET /auth/me	Logout or retrieve current account
GET PATCH /portfolio/me	Read or update owner profile
GET POST /portfolio/me/links; PATCH DELETE /portfolio/me/links/{link_id}	Manage external portfolio links
GET POST /portfolio/me/education; PATCH DELETE /portfolio/me/education/{education_id}	Manage school degree dates coursework
GET POST /portfolio/me/skills; PATCH DELETE /portfolio/me/skills/{skill_id}	Manage name category proficiency

Endpoint	Contract and purpose
GET POST /portfolio/me/experience; PATCH DELETE /portfolio/me/experience/{experience_id}	Manage experience records
GET POST /portfolio/me/certificates; PATCH DELETE /portfolio/me/certificates/{certificate_id}	Manage certificate records; SQL fields verified in Section 1.11; API validation remains to be tested
GET POST /portfolio/me/job-preferences; PATCH DELETE /portfolio/me/job-preferences/{preference_id}	Manage private job preferences; SQL fields verified in Section 1.11; API validation remains to be tested
GET POST DELETE /portfolio/me/share	Read enable or disable sharing
GET /projects	Public listing; search category limit offset parameters
POST /projects; GET /projects/mine	Create owner project or list owned projects
GET PATCH DELETE /projects/{project_id}	Read update or delete a project subject to access rules
POST /projects/{project_id}/images; POST /projects/{project_id}/images/url	Upload image or attach an image URL
POST /projects/{project_id}/links; DELETE /projects/{project_id}/media/{media_id}	Add link or remove project media
PUT DELETE /projects/{project_id}/model	Multipart model upload replacement or removal
POST GET /guidance; GET /guidance/latest	Generate analysis list history retrieve latest
GET /public/portfolios/{share_token}; GET /public/portfolios/{share_token}/projects/{project_id}	Read permitted shared portfolio or project
GET /health	Backend health status

Project create/update fields are title, description, content_blocks, project_date, subject, technologies and display_order. Responses add id, images, links, model_3d and timestamps. GET /projects supports keyword matching across title, description, subject and technologies; every keyword must match somewhere. category matches subject case-insensitively; limit is 1–100 with default 20 and offset supports pagination. Unsupported uploads return 400, oversized uploads 413 and validation errors 422. Authentication and ownership failures must be handled by the client.

4.6 3D Model Architecture

A physical Raspberry Pi was captured in Scaniverse and exported as GLB. An authenticated multipart request to PUT /projects/16/model validates the file and stores it in Supabase Storage. project_models stores the URL, file format, filename, byte size, storage path and project association. A project response exposes model_3d.url. The frontend normalizer passes that URL to ModelViewer; the viewer loads the asset and renders it with browser controls for rotation and zoom. This keeps binary models outside PostgreSQL.

The six-step UML 3D flow includes future frontend file selection. The M2 upload was performed in Swagger UI. Rendering and model manipulation occur in the normal deployed React application.

4.7 Deployment and Integration

Vercel serves the frontend, Render serves FastAPI, and Supabase provides persistent database and file storage. The frontend API base must target Render, the backend CORS allowlist must include the Vercel origin, and frontend deep links require the SPA rewrite. The team resolved production configuration and CORS issues during integration. Store credentials only in backend configuration. Render local file storage is unsuitable for persistent uploads; production uses Supabase Storage.

Team validation covered deployed search, authenticated creation and editing, and cloud-hosted model viewing. Production URLs and repeatable evaluation steps are provided in Appendix A.

4.8 Proposed M5 Chatbot Roadmap and Milestone Architecture

This is a proposed design for R18, not part of the implemented M2 schema or a verified deployment. React will provide chat, proposal review, editable plan and completion views. FastAPI will authenticate the student, load the owned portfolio and current plan, call the configured AI service and validate a structured proposal. Provider credentials remain on the backend.

Proposed flow: student request → backend portfolio context → AI proposal → validated preview → student approval → transactional database save → refreshed roadmap and progress. Generation alone must not modify the saved plan. Rejection leaves the saved plan unchanged. Version checks should prevent approval of a stale proposal from overwriting newer edits.

Proposed relational entities: career_roadmaps (id, portfolio_id, title, status, version, created_at, updated_at); roadmap_goals (id, roadmap_id, title, description, target_date, display_order, created_at, updated_at); goal_milestones (id, goal_id, title, description, due_date, status, completed_at, display_order, created_at, updated_at). IDs and ownership foreign keys should follow the current integer-key pattern. A roadmap has multiple goals, and a goal has multiple milestones. Existing career_guidance remains generated analysis, not editable completion state.

Proposed supporting records: roadmap_proposals should retain the owner, base plan version, validated proposed changes and pending/approved/rejected status with approval time. Chat history storage and retention remain a design decision; retain only the context needed for continuity. These entities require reviewed migrations and are not additions claimed in the current twelve-table schema.

Proposed API responsibilities include generating a proposal, retrieving it for review, explicitly approving or rejecting it, owner CRUD for roadmaps/goals/milestones, and updating completion status. Route names are to be finalized; these operations are not existing M2 API endpoints. Every write must resolve ownership through portfolio → roadmap → goal → milestone. Approved changes and proposal status should commit atomically.

Completion progress can be computed from completed milestones divided by total milestones, with an empty plan shown as having no milestones. The student controls completion state; AI suggestions must not mark work complete automatically. M5 acceptance covers approval/rejection, persistence, individual milestone edits, authorization failures, stale versions, AI failures and database-save errors.

5 High Level UML Diagrams

5.1 Overview

Nuradil prepared one class diagram and six sequence diagrams. The following images are the original PNG exports from milestones/m2/uml/exports in the repository; editable PlantUML sources are retained in milestones/m2/uml. They describe the high-level design. The class diagram uses UUID identifiers and conceptual methods, whereas the inspected SQLAlchemy implementation uses integer IDs and endpoint/service functions. Use the field definitions in Sections 1 and 4 for implementation naming.

Reconciliation notes: ThreeDModel corresponds to ProjectModel and project_models; biography maps to bio; careerInterests is a list; shareUrl is derived from sharing settings and token. The class diagram omits portfolio links and the certificate and job-preference tables. Sequence diagrams show intended flows and do not establish that frontend model uploading or live AI guidance is complete.

5.2 High Level Class Diagram

User owns Portfolio; portfolio aggregates education skills experience projects and guidance. Projects contain media and at most one 3D model. AIService represents the guidance boundary. Update the conceptual identifier types and names during final architecture reconciliation.

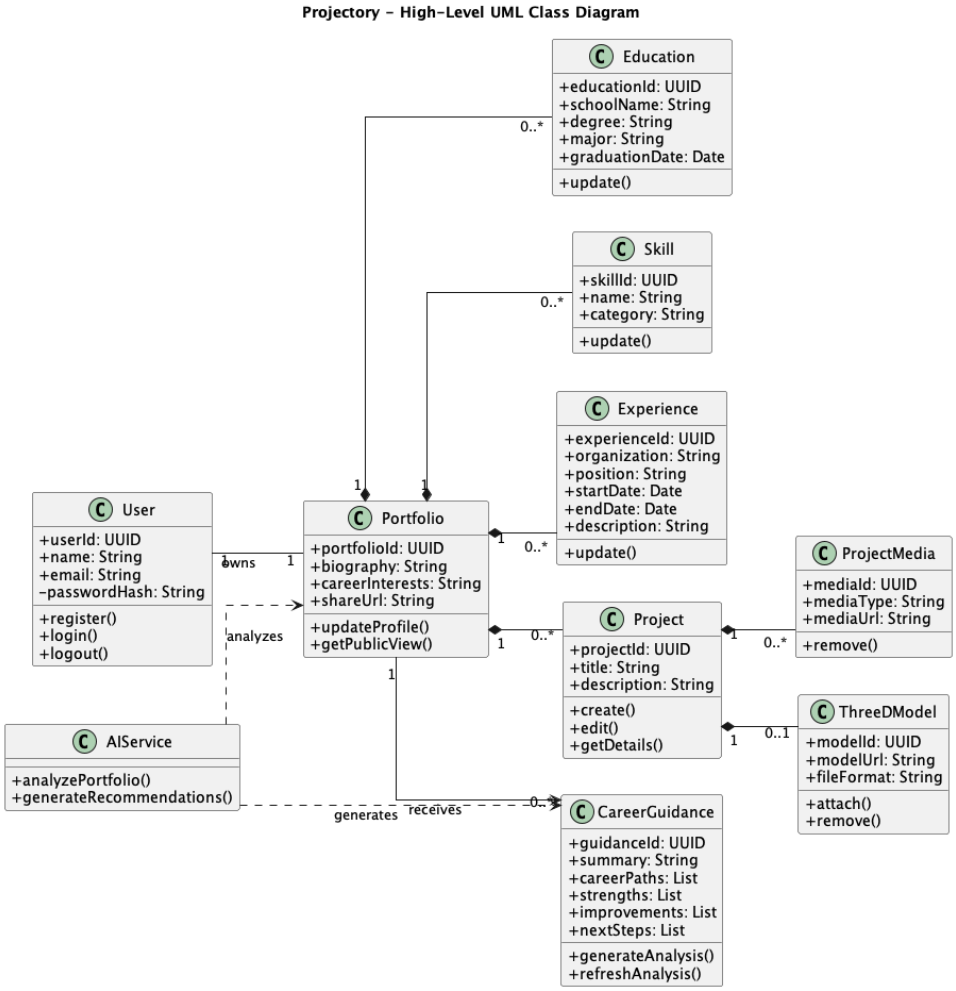


Figure 5.1 Original team UML export class-diagram.png

5.3 Registration and Login

The student submits registration or login through React. FastAPI validates the request, accesses the user record and returns the authentication result. Invalid input and credentials must return a clear error.

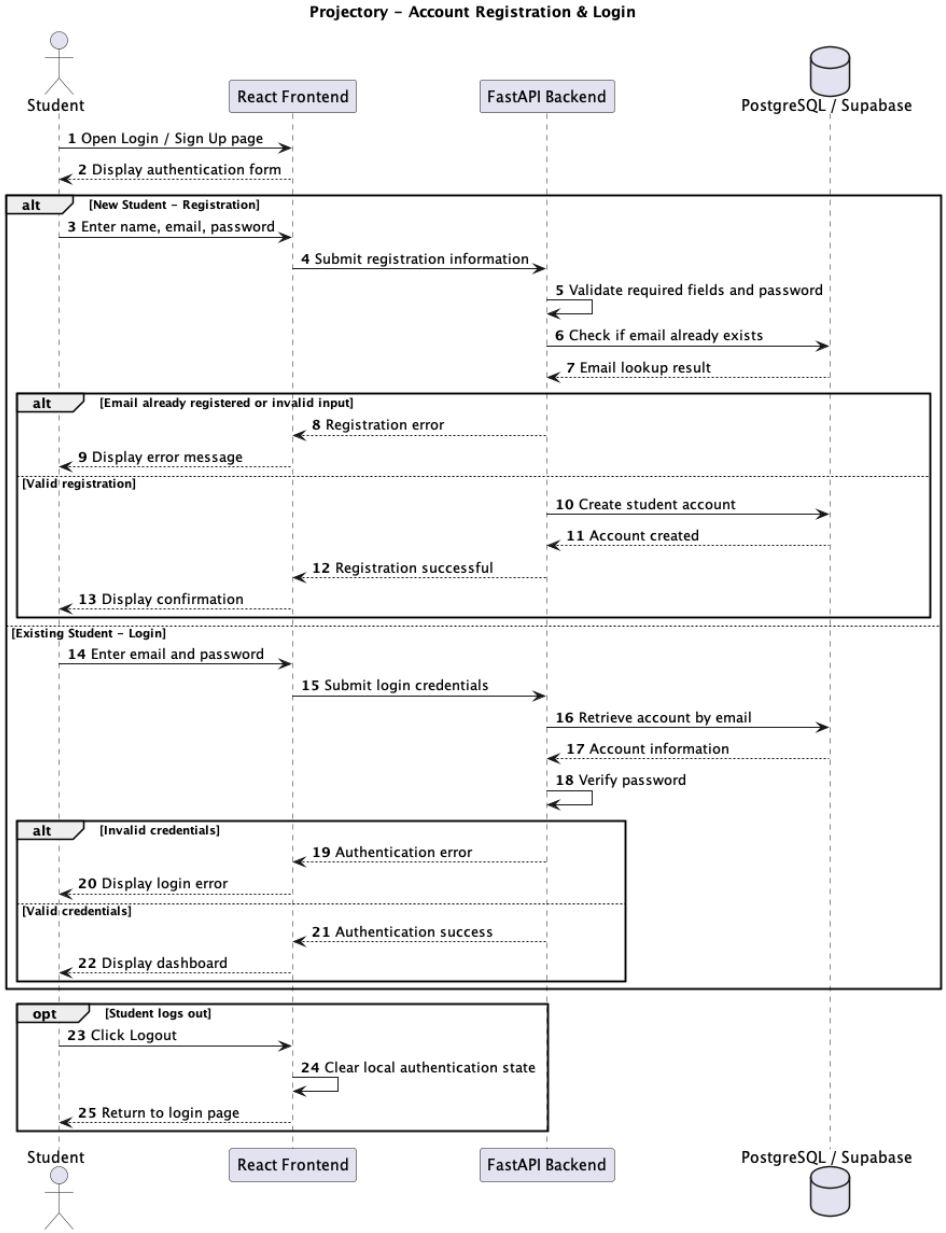


Figure 5.2 Original team UML export sequence-auth.png

5.4 Portfolio Creation and Management

The owner retrieves and updates portfolio information through React and FastAPI. Ownership governs the permitted database changes; saved values are returned to the frontend.

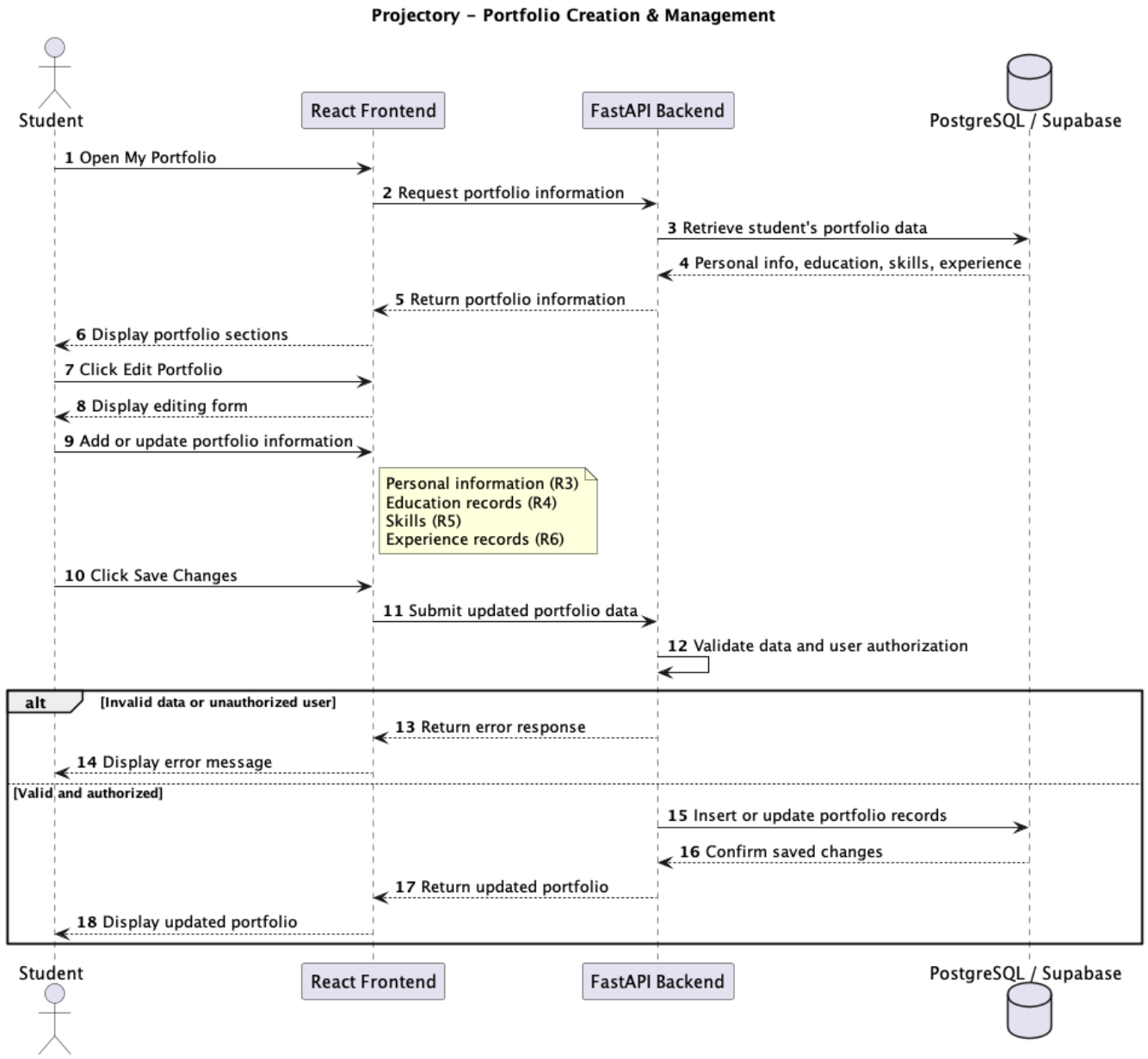


Figure 5.3 Original team UML export sequence-portfolio.png

5.5 Project Creation and Management

React submits project information to FastAPI, which validates ownership and input, saves the record and returns the stored project for display.

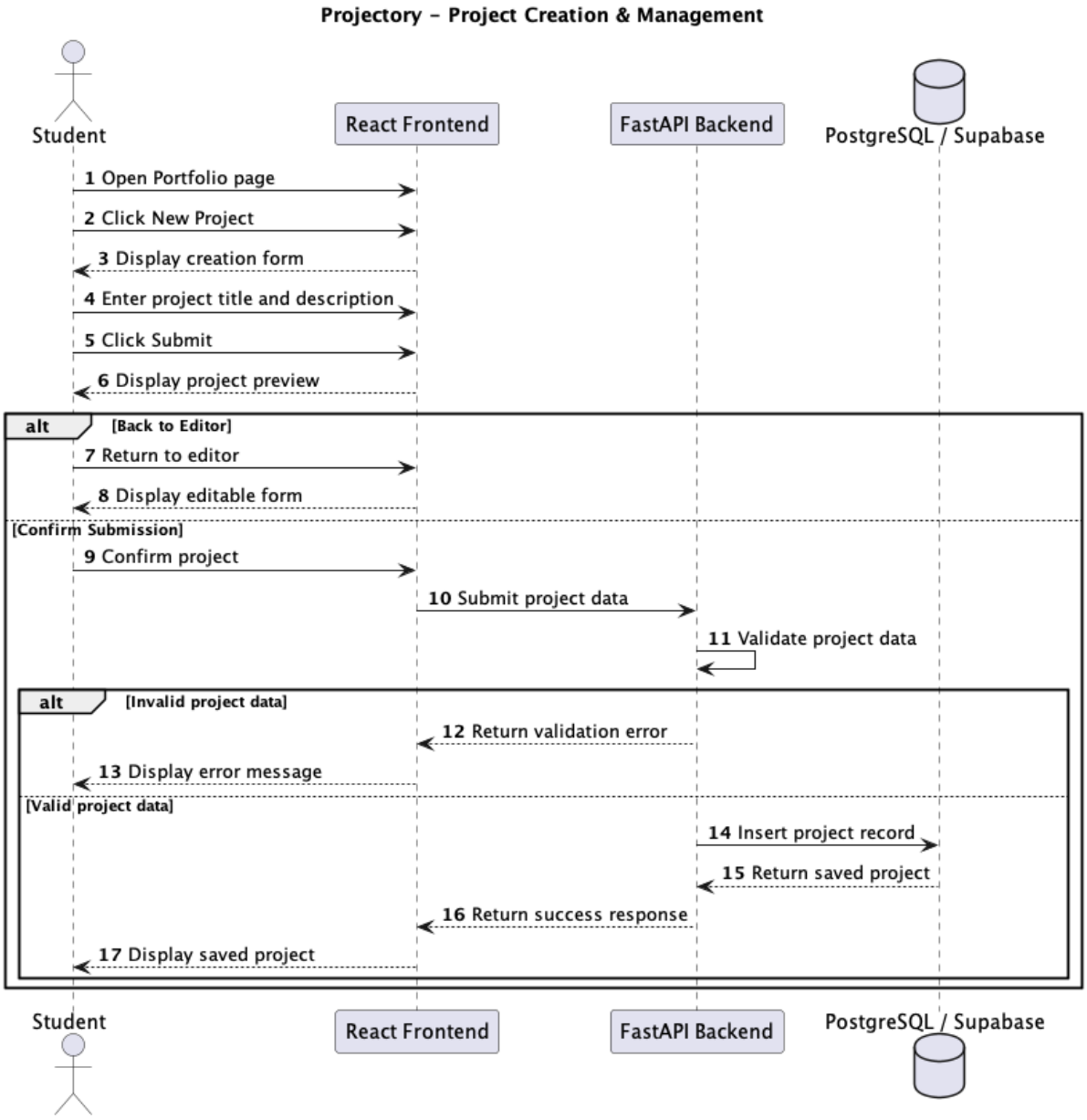


Figure 5.4 Original team UML export sequence-project-creation.png

5.6 Portfolio Viewing and Sharing

The owner obtains a shareable reference. A visitor requests the public portfolio, and the backend checks sharing before returning permitted read-only content.

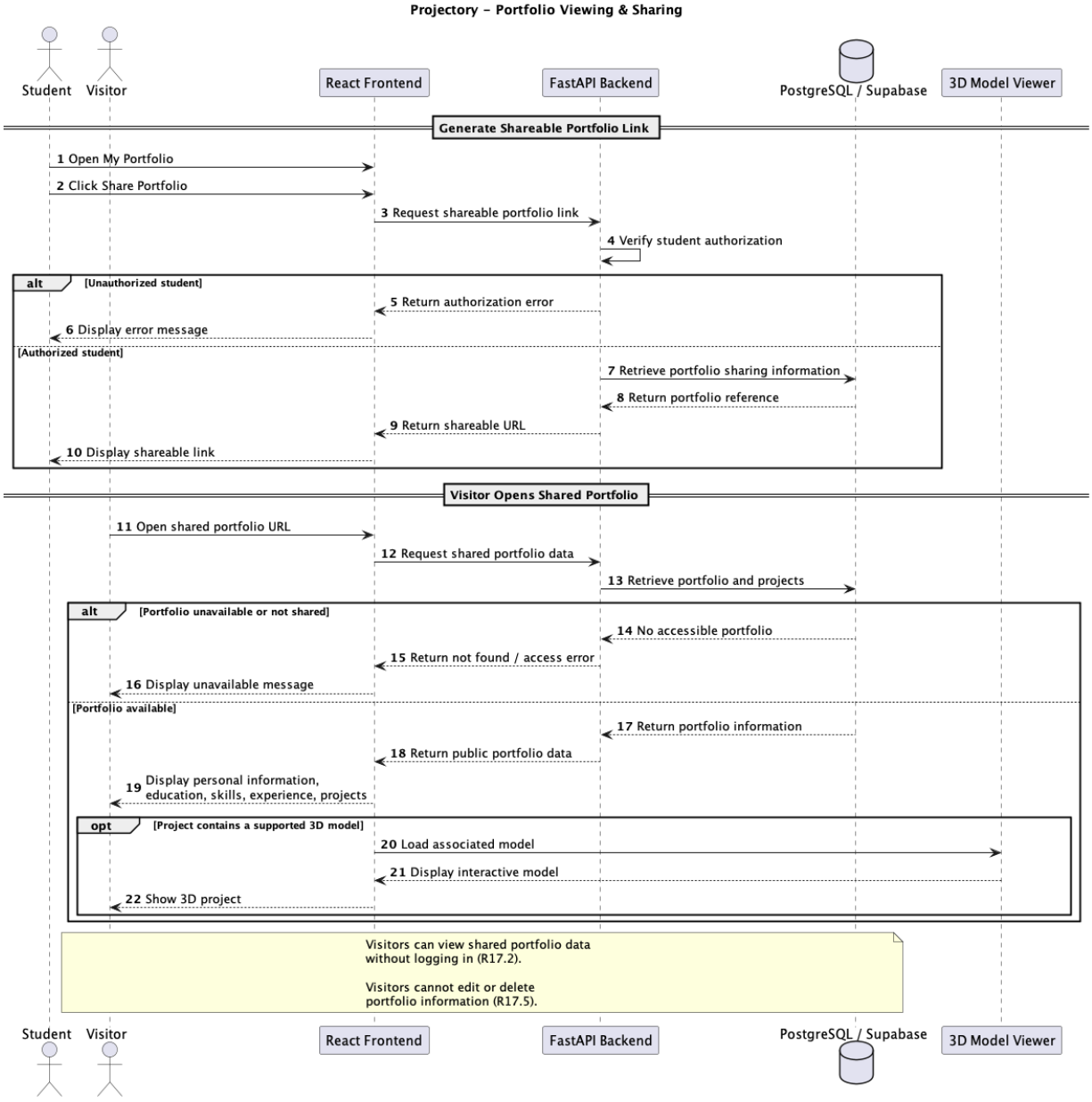


Figure 5.5 Original team UML export sequence-sharing.png

5.7 Interactive 3D Project Showcase

FastAPI validates an authorized model attachment, stores the file and associates metadata with a project. The viewer retrieves the model URL and loads it from storage for rotation and zoom. Frontend file selection is planned; M2 used Swagger for upload.

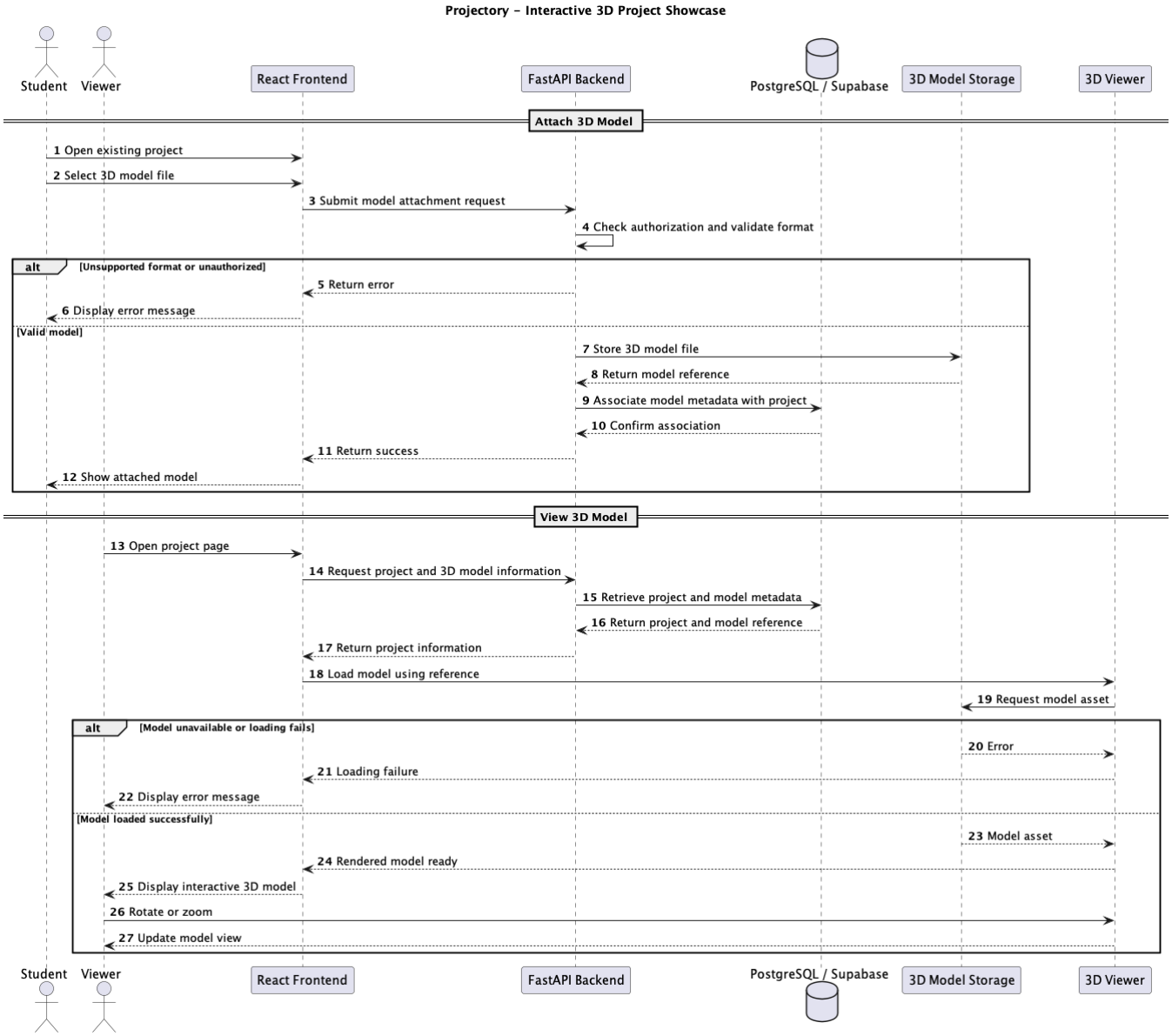


Figure 5.6 Original team UML export sequence-3d.png

5.8 AI Career Guidance

The planned workflow retrieves portfolio data, requests structured guidance, stores the result and returns it to the student. Complete live external AI generation remains unverified and is part of the M5 plan.

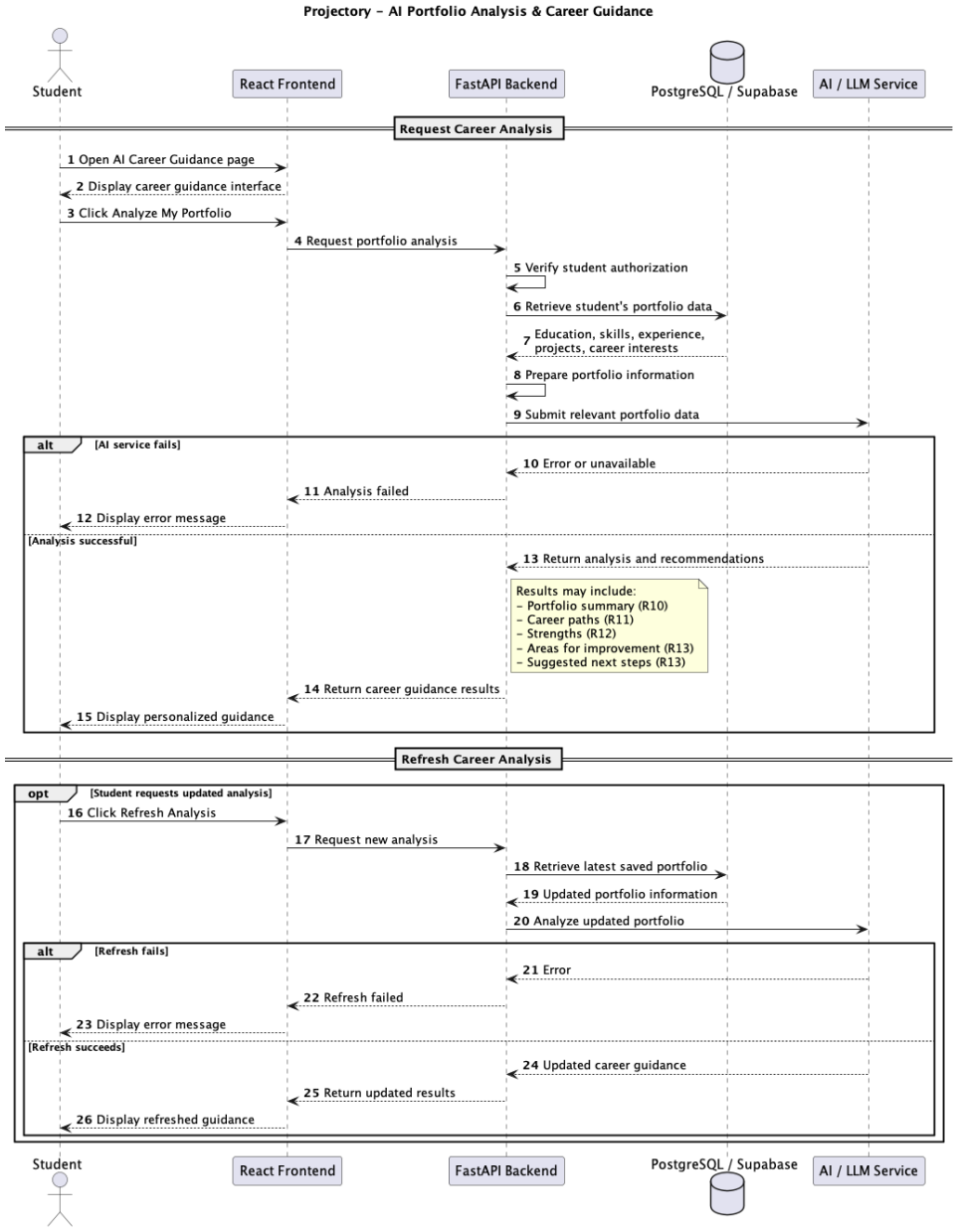


Figure 5.7 Original team UML export sequence-ai.png

6 Key Risks

6.1 Schedule and Integration Risk

Risk: Parallel frontend, backend, AI and 3D work produced late API handoffs and overlapping changes. Combining components close to the demo left little review time.

Mitigation: Agree field contracts before coding, use focused pull requests and test the deployed vertical slice after each integration. Rohit coordinates with Ian and Ali; scope optional work after core flows are stable.

6.2 AI Integration and Skills Risk

Risk: Live portfolio analysis has not been verified. Provider setup, structured response handling and unfamiliar tools can delay reliable guidance.

Mitigation: Obaidullah and Ali will study the existing provider and schema contracts, test one portfolio end to end, validate output shape and handle unavailable services. Identify mock results separately from external AI results.

6.3 3D Compatibility and Performance Risk

Risk: Scan quality, file complexity and camera framing can affect rendering. Frontend upload is incomplete, so students cannot yet attach a model in the normal UI.

Mitigation: Ian, Ali and Rohit will add upload controls, enforce existing format/size validation, test several GLB assets and provide loading and error states. Retain project text when rendering fails.

6.4 Deployment and Availability Risk

Risk: CORS configuration and a stale deployment branch affected integration. Free-tier backend startup delays can interrupt demonstrations.

Mitigation: Nuradil and Ali will verify deployment branches, API base URLs, allowed origins and storage settings before each review. Document restart and deployment steps and test the production search flow.

6.5 Authentication and Privacy Risk

Risk: Portfolio ownership, public project discovery and public storage URLs introduce possible disclosure paths. Visibility behavior for user-created projects needs review.

Mitigation: Ali and Ian will test requests from owner, other account and visitor; confirm excluded private fields and sharing disable behavior. Review public object access separately from portfolio visibility.

6.6 Team Coordination Risk

Risk: Different schedules and late handoffs caused uncertainty about integration ownership and unfinished documentation.

Mitigation: Rohit and Fanta will share named tasks, blockers and acceptance criteria after each meeting. Use GitHub issues with owners and review dates for M5, and update the shared plan when assignments change.

6.7 Licensing and Content Risk

Risk: Third-party images, scanned objects and dependencies may carry copyright or redistribution conditions.

Mitigation: Nuradil will record dependency licenses and the source of demonstration assets. The Raspberry Pi scan was captured by the team lead in Scaniverse; verify all other media is owned or permitted before distribution.

7 Project Management

Projectory's M2 work was divided by role: Ian developed the React interface, Ali implemented FastAPI and Supabase integration, Nuradil maintained repository and UML materials, Obaidullah contributed requirements and AI planning, Fanta supported coordination, and Rohit coordinated integration, 3D testing and documentation. Progress and blockers were shared through Discord and team discussions; GitHub provided the shared code history, branches and pull requests.

The team tested production workflows as integration progressed, resolving API and deployment configuration issues before the demonstration. Late handoffs and overlapping work showed the need for earlier contract review and clearer completion criteria. No dedicated Jira or Notion board is confirmed for M2. For M5, Fanta and Rohit propose GitHub issues with an owner, target review date and acceptance check for each task, updated after scrum discussions so every member can see progress and changed assignments. Instructor feedback will be linked to those tasks and reviewed before final submission.

Appendix A Vertical Software Prototype Delivery Report

A.1 Prototype Objective

The M2 vertical prototype exercises the complete stack from the browser through middleware to persistent database records. Projectory demonstrates database-backed listing, search, authenticated creation and updates. It also demonstrates a 3D asset stored separately in cloud storage and rendered inside the project view.

A.2 Deployed Application

Frontend: https://sfsu-csc648-06-fall26-team03.vercel.app/

Explore test page: https://sfsu-csc648-06-fall26-team03.vercel.app/explore

Backend API documentation: https://careergps-backend-n1uj.onrender.com/docs

GitHub repository: https://github.com/Nuradil22/SFSU-csc648-06-Fall26-team03

Database schema: https://github.com/Nuradil22/SFSU-csc648-06-Fall26-team03/blob/main/app/backend/schema.sql

A.3 Implemented Prototype Features

Database Backed Listing and Search

The Explore page retrieves stored projects through GET /projects. The team reported twelve seeded records for M2, covering four demonstration students. Searching robot returned Line Following Robot. The included listing and search screenshots show the frontend results. Seeded records are demonstration content stored in the database, rather than frontend hardcoded project arrays.

Authentication and Persistent Project Management

The team registered and authenticated a user, created a Raspberry Pi project with ID 16 and edited its title, description and subject through the deployed frontend. An authenticated GET /projects/mine response retrieved the saved record, confirming persistence. The model demonstration later displayed the revised title Raspberry Pi Interactive 3D Scan.

Interactive 3D Project Showcase

Rohit captured the Raspberry Pi using Scaniverse and exported a GLB. PUT /projects/16/model returned HTTP 200 with model ID 3, file_format glb, original_filename Scaniverse 2026-10-19 194713.glb, file_size_bytes 2217516 and creation time 2026-10-20T18:21:28.340534Z. The asset was stored in Supabase Storage. The deployed frontend loaded it into the interactive viewer; Rohit confirmed that mouse rotation and zoom worked. No authentication tokens are reproduced in this report.

A.4 Deployment Validation

The results below record team testing and the October 20 demonstration. They are not a claim that the document preparation process reran the live tests.

Test	Recorded M2 result
Frontend loads on Vercel	Passed in team testing
API documentation accessible	Passed in team testing
Database listing and robot search	Passed; Figures 3.6 and 3.7

Test	Recorded M2 result
Registration and login	Passed in team testing
Authenticated project creation	Passed; Figure 3.5
Edit title description subject and retrieve saved data	Passed in team testing and saved response
Backend GLB upload	Passed; recorded HTTP 200 response
Supabase model storage	Passed; returned URL and metadata
Interactive model rendering rotation and zoom	Passed; Figure 3.8 and user confirmation
Direct GLB upload through frontend	Not yet implemented
Complete external AI analysis	Not yet verified
Public sharing and owner isolation edge cases	Further end-to-end validation required

Evaluation Steps

1. Open the Explore test-page URL and wait for the backend if it is starting. Confirm project cards load from stored data.

2. Search robot and confirm Line Following Robot appears; clear the search and try a category filter.

3. Register or sign in using an evaluator-owned account. Complete Account Setup, create a project and save it.

4. Edit the project title, description and subject; reload or sign in again and confirm saved values remain.

5. For a project owned by the evaluator, authorize Swagger using the same account and upload a supported GLB through PUT /projects/{project_id}/model. Use that actual project ID.

6. Open or refresh the project in the frontend, expand the card and verify that its model rotates and zooms. Record any mismatch or load failure.

Project 16 is the team demonstration record and is not a substitute for evaluator ownership. No private credentials or bearer tokens are included. Public visibility of the demonstration portfolio must be confirmed before relying on anonymous access.

A.5 Technical Lessons Learned

The prototype validated the deployment architecture and helped the team integrate independently developed components. The main lessons were agreeing API field names, handling authenticated requests, confirming CORS and production branch configuration, verifying data persistence and storing binary assets outside the relational database. Backend handoff documentation and interactive API docs help the team understand the shared contract. Frontend upload controls and complete AI guidance remain the next integration steps.

A.6 Instructor Feedback

At the October 20, 2026 demonstration, the professor asked how AI career guidance would be fleshed out and how students would use portfolios. These questions requested clearer product behavior and use cases; they do not establish that the proposed features were already implemented.

The team described an AI chatbot that helps a student create and edit a career roadmap and individual goals or milestones. The student reviews and approves proposed changes before they are saved, and can track milestone completion. This is planned M5 functionality, represented by proposed R18 and Section 4.8.

The team also discussed portfolio use cases beyond computer science in other STEM fields. Illustrative use cases include engineering design projects and prototypes, biology or chemistry laboratory research and findings, and mathematics or statistics analyses. Students can present methods, skills, results and supporting images, links or suitable 3D models. These examples clarify intended use; they are not claims of completed field-specific features or user validation.

A.7 Team Response and M5 Implementation Plan

The team response is to define an actionable guidance workflow with student approval and completion tracking, and clarify non-CS STEM portfolio use cases. The following work and owners are proposed for M5; assignments and dates need team scheduling. No exact M5 calendar date is asserted here.

Priority	Planned work	Proposed owner and completion check
High	Live AI guidance and proposed R18 roadmap workflow	Obaidullah Ali Ian; guidance persists, approved plans save, rejected proposals do not write, owner completion tracking survives reload; target M5
High	Frontend 3D upload	Ian Ali Rohit; supported file attaches to owned project and renders after reload
High	Portfolio end-to-end testing	Ian Ali Rohit; create edit delete and persistence verified across sections
High	Visibility and public sharing	Ian and Ali; owner other user and anonymous access checks pass
Medium	Validation and error handling	Frontend and backend; invalid input and unavailable service tests produce clear messages
Medium	Accessibility and responsiveness	Ian; keyboard and small-screen walkthroughs completed
Medium	Tests and deployment documentation	Ali and Nuradil; repeatable core-flow checks and updated setup guide

Feedback follow-up: Obaidullah and Ali are proposed to define and validate the chatbot and roadmap contracts, with Ian implementing review, approval and tracking views. Acceptance requires an approved plan to persist after reload, a rejected proposal to leave saved data unchanged, and owner-only goal and milestone updates. Rohit and Ian are proposed to review representative non-CS STEM portfolios before M5 and confirm that methods, results and supporting media can be presented clearly. These are planned checks, not completed tests.

A.8 Conclusion

The M2 prototype demonstrated a deployed React frontend connected to FastAPI, Supabase PostgreSQL and Supabase Storage. Project discovery, persistent project management and interactive 3D viewing establish the foundation for completing the remaining Projectory functionality by M5.